# Restaurant Order Management System - Final Report

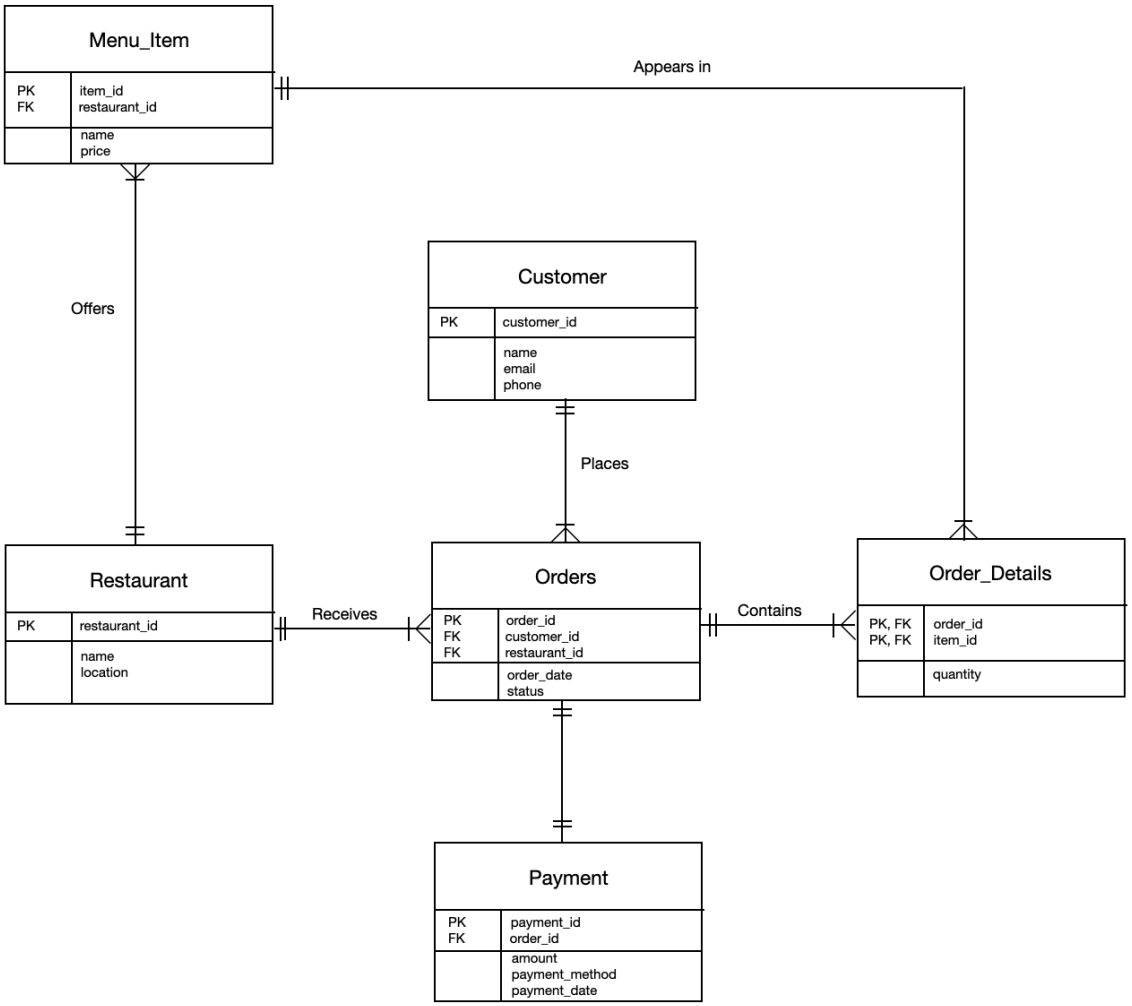
## 1. Summary

This project is about the design and implementation of a database-backed application for managing restaurant orders. The system is developed using Oracle and demonstrates core database concepts including relational modeling, normalization, and data integrity. The database supports operations such as storing customer information, managing restaurant menus, processing orders, and recording payments.

## 2. Introduction

In modern restaurant operations, managing customer orders efficiently is important for maintaining service quality and operational accuracy. Manual systems or poorly designed databases can lead to redundancy, inconsistency and errors. The purpose of this project is to manage restaurant operations including customers, orders, menu items, and payments in a structured database. The system should ensure that all data is stored efficiently, relationships are properly maintained, and queries can be executed effectively.

## 3. Data Modeling



The database is designed using an Entity-Relationship (ER) model that identifies the main entities and their relationships.

***3.1. Primary entities:***

- **Customer**: stores information about customers who place orders. It includes attributes such as customer_id, name, email, and phone number. Each customer is uniquely identified by a primary key.
- **Restaurant**: represents different restaurants in the system. It contains attributes such as restaurant_id, name, and location. This allows the system to manage multiple restaurants independently.
- **Menu_Item**: stores the items offered by each restaurant. It includes item_id, name, price, and a foreign key referencing the restaurant. Each menu item belongs to exactly one restaurant.
- **Orders**: records each transaction made by customers. It includes attributes such as order_id, customer_id, restaurant_id, order date, and status. Each order is associated with one customer and one restaurant.
- **Order_Details**: this is an associative entity that represents the relationship between orders and menu items. It contains a composite primary key consisting of order_id and item_id, along with a quantity attribute. Each record represents a specific item within a specific order.
- **Payment**: stores information about transactions related to orders. It includes attributes such as payment_id, order_id, amount, payment method, and payment date. Each payment is linked to a specific order.

***3.2. Relationships:***

The relationships between entities define how data is connected within the system. These relationships are represented in the ER diagram using appropriate cardinalities.

- **Customer to Orders (1:M)**: A single customer can place multiple orders, while each order is associated with exactly one customer. This relationship ensures that customer activity can be tracked over time.
- **Restaurant to Orders (1:M)**: A restaurant can receive many orders, but each order is linked to only one restaurant. This allows the system to manage orders across multiple restaurants.
- **Restaurant to Menu_Item (1:M)**: Each restaurant can offer multiple menu items, while each menu item belongs to one restaurant. This relationship organizes menu data by restaurant.

- **Orders to Order_Details (1:M)**: An order can include multiple items, represented as multiple records in the Order_Details table. Each record corresponds to one item within the order.
- **Menu_Item to Order_Details (1:M)**: A menu item can appear in many different orders. This relationship allows the system to track how frequently items are ordered.
- **Orders to Payment (1:1):** Each order is associated with a single payment record, and each payment corresponds to one order. This ensures accurate tracking of financial transactions.

## 4. Design Decisions

Several important design decisions were made to ensure that the database is efficient, scalable, and adheres to normalization standards. One of the key decisions was the introduction of the Order_Details table to resolve the many-to-many relationship between Orders and Menu_Item. Without this table, it cannot cleanly represent multiple items within a single order.

Another significant decision was the use of primary keys and foreign keys to enforce referential integrity. Each table includes a primary key that uniquely identifies its records, while foreign keys establish relationships between tables. For example, the Orders table references both Customer and Restaurant, ensuring that every order is linked to valid entities.

The database is normalized to the Third Normal Form (3NF). First Normal Form (1NF) is achieved by ensuring that all attributes contain atomic values with no repeating groups. Second Normal Form (2NF) is satisfied by removing partial dependencies, particularly in the Order_Details table where the composite primary key fully determines all attributes. 3NF is achieved by removing transitive dependencies, ensuring that non-key attributes depend only on the primary key. For instance, menu item prices are stored only in the Menu_Item table and not duplicated elsewhere.

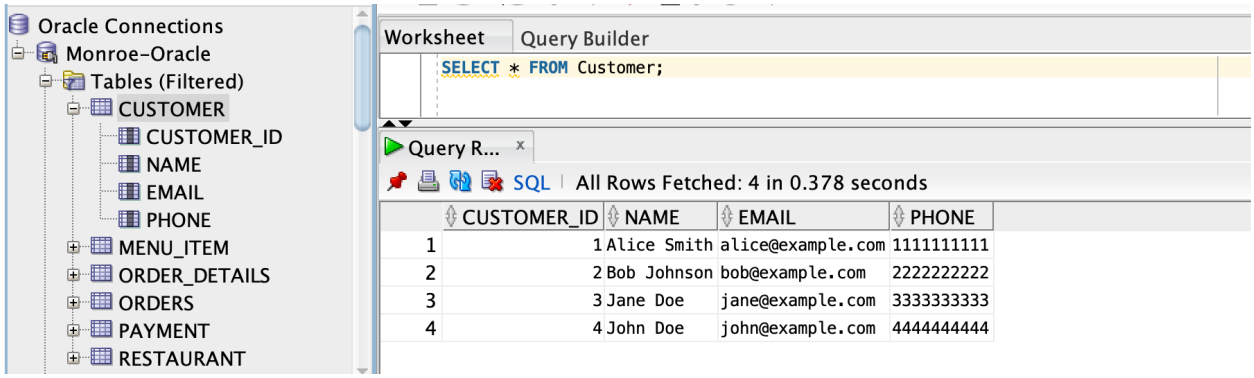
These design choices help reduce redundancy, improve data consistency, and make the database easier to maintain.

## 5. Implementation and Results

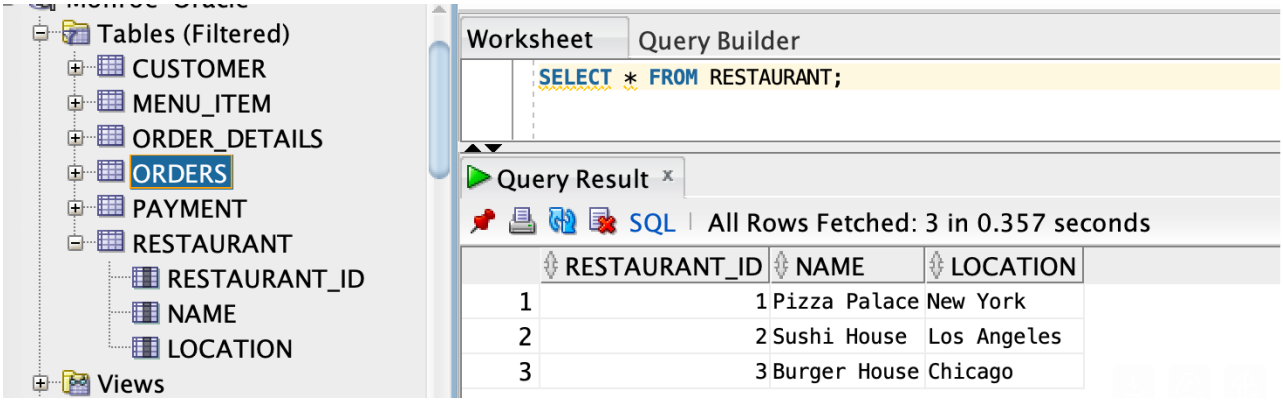
The database was implemented using Oracle SQL. The schema was created using Data Definition Language (DDL) statements, and sample data was inserted into each table to simulate real-world usage. The implementation was supported by 3 SQL files:

- **schema.sql:** defines the relational database schema using DDL statements, including the creation of tables, primary keys, foreign keys, and constraints.
- **sample_data.sql:** contains INSERT statements used to fill tables with representative data.
- **reset.sql:** includes DROP TABLE statements that allow the database to be reset by removing all existing tables and constraints.

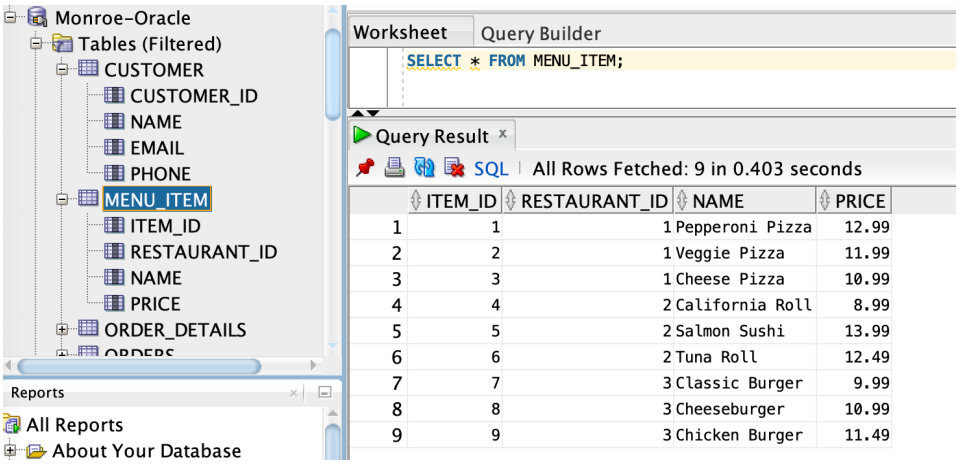
Additionally, the following screenshots illustrate the structure and sample data of each table in the database. Each table includes its attributes along with representative records to demonstrate how the system operates.



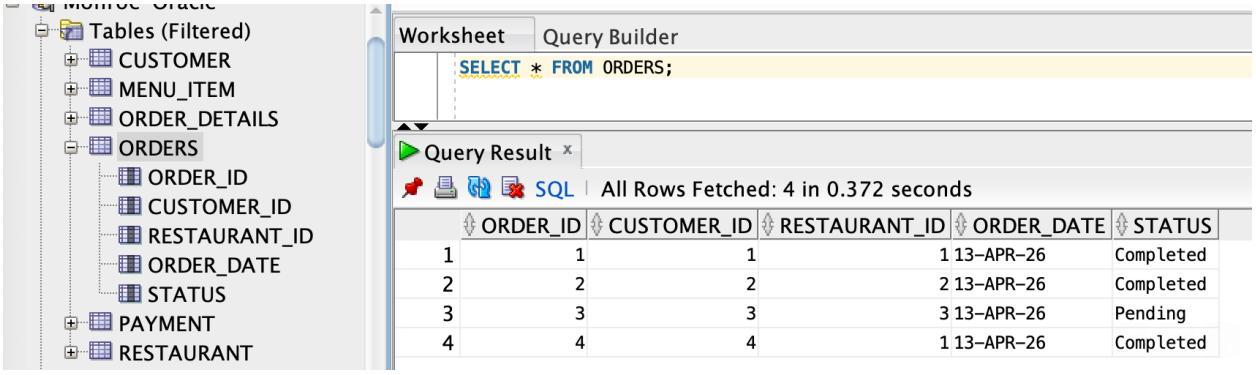

Customer table containing customer attributes, with customer_id as the primary key and sample records demonstrating stored customer information.



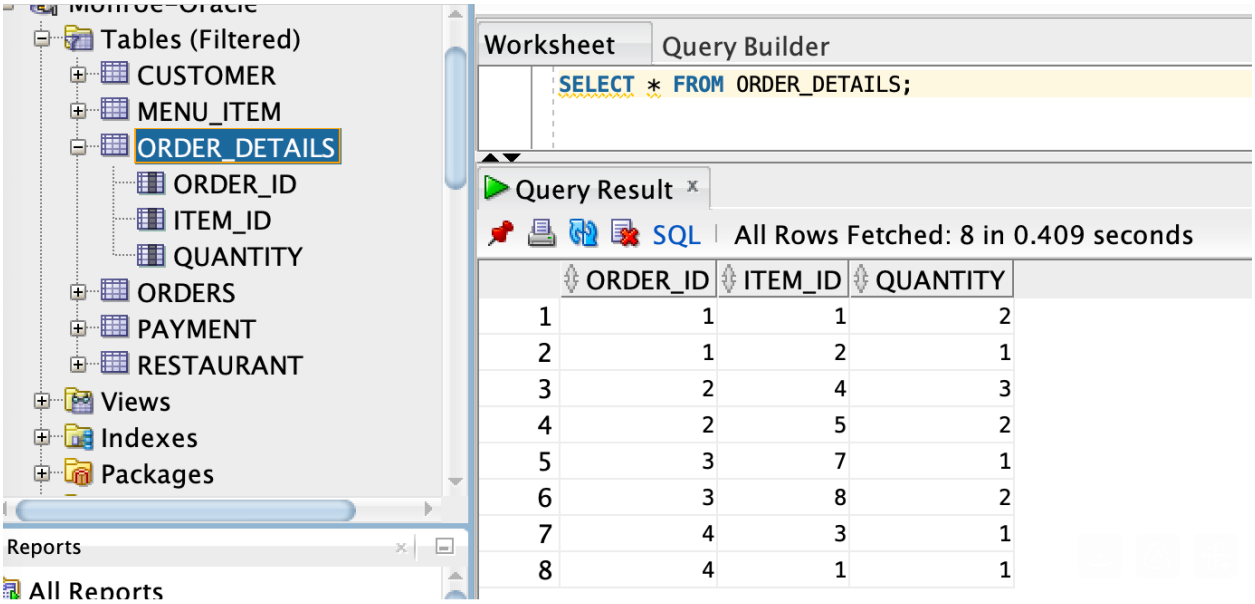

Restaurant table containing restaurant information with auto-generated IDs, name and location.



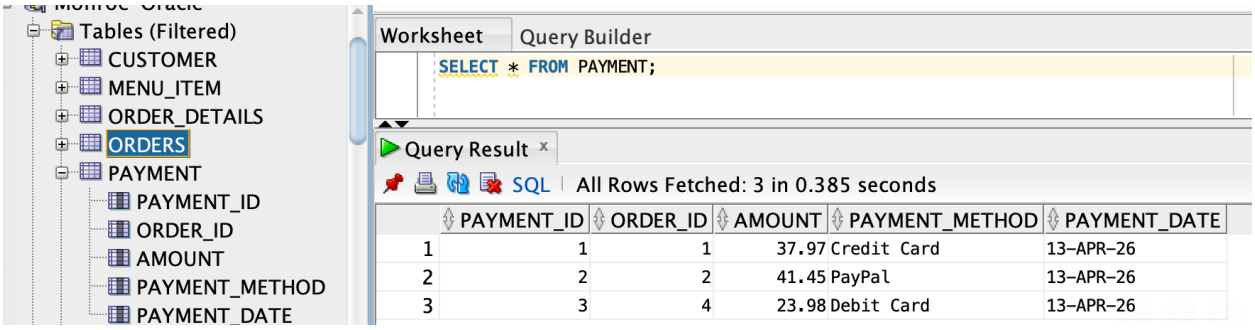

Menu_Item table structured with item_id as the primary key and restaurant_id as a foreign key, establishing a relationship between menu items and their respective restaurants.




Orders table containing order_id as the primary key and foreign keys customer_id and restaurant_id, linking each order to a specific customer and restaurant.



Order_Details table designed with a composite primary key (order_id, item_id) and foreign keys referencing Orders and Menu_Item, effectively resolving the many-to-many relationship between orders and items while storing quantity.




Payment table recording transaction details for each order, including payment method, amount, and date which is generated by system date.

## 6. Conclusion

In conclusion, this project successfully demonstrates the design and implementation of a database for a restaurant order management system. The database structure follows best practices in normalization and relational modeling, ensuring data integrity and efficient storage. The system can be further extended with additional features such as delivery tracking, employee management, or customer reviews. Overall, it provides a strong foundation for understanding database systems and their application in real-world scenarios.